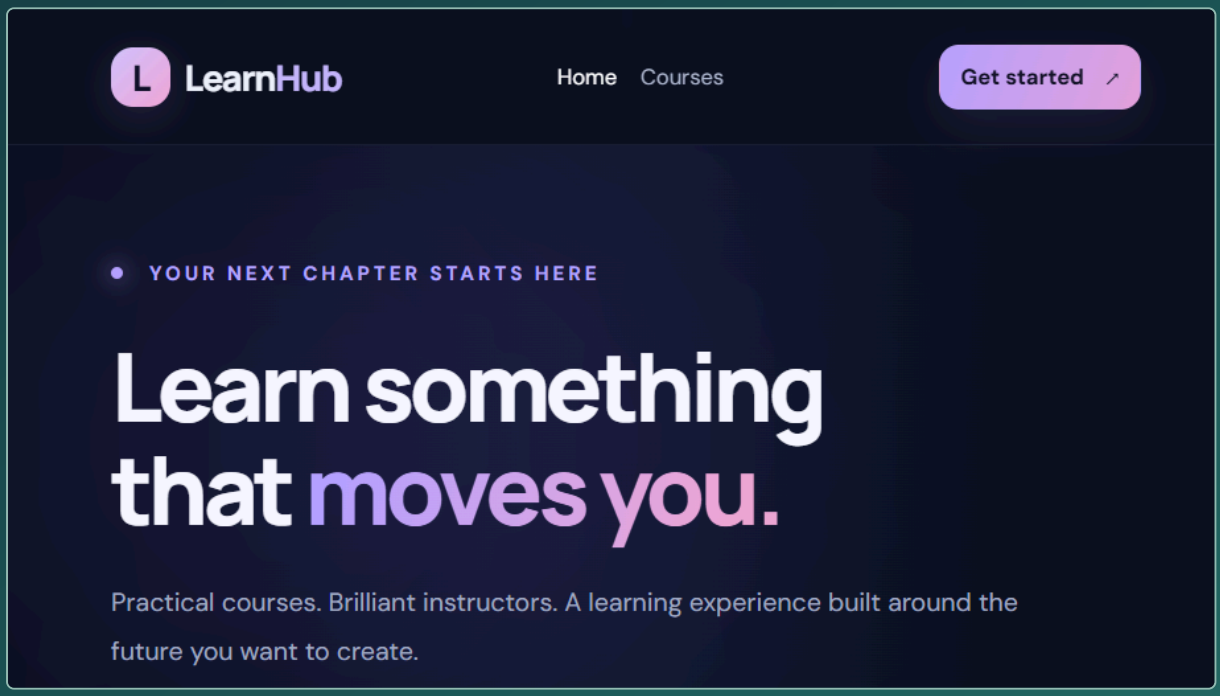
MERN STACK · LOCAL QA EVIDENCE

LearnHub Final Project Report

A verified walkthrough of the running learning platform, its API contracts, role controls, database connection, and source implementation.

React + Vite | Node + Express | MongoDB + Mongoose | JWT



Verification date: 17 October 2026
Frontend: http://localhost:5173
API: http://localhost:5000/api
Database: local MongoDB database `learnhub`; credentials and JWT secret are intentionally omitted.

Executive Summary

The project was run against the local MongoDB service and verified through its browser UI and HTTP API. Evidence below was captured from the actual running application and real endpoint responses; no demo/mock screenshots were substituted.

24	4	3	0
API CHECKS PASSED	SEEDED COURSES AFTER CLEANUP	ROLE DASHBOARDS VERIFIED	SECRETS INCLUDED

> **Local run status:** MongoDB connected; backend health returned `ok`; frontend returned HTTP 200; frontend production build completed successfully.

Review changes verified

The React JSX runtime initially failed because Vite had no React plugin configuration. `frontend/vite.config.js` now activates the installed plugin, and the production build passes. Existing MongoDB course documents include a legacy `instructorName` field while the populated instructor reference can be null; the course card and detail view now fall back to that stored name. The existing database documents were not migrated.

> **Remaining notes:** the build succeeds but Vite reports a JavaScript chunk slightly above 500 kB. Dependency installation previously reported 3 high-severity backend and 2 moderate-severity frontend advisories; review `npm audit` before deployment. The backend terminal output was verified, but a genuine desktop screenshot of that specific terminal could not be selected in this environment, so no unrelated terminal image is presented as evidence.

Setup and Database

Requirements and commands

Verified environment: Node.js 24.18.0, npm 11.16.0, MongoDB 8.3. The backend package script uses nodemon for development. Run the commands from the project root in separate terminals:

```
cd backend
npm install
npm run dev

cd frontend
npm install
npm run dev
```

Setting	Verified value or handling
`PORT`	5000
`MONGO_URI`	`mongodb://127.0.0.1:27017/learnhub` (local development DB)
`JWT_SECRET`	Configured in local `backend/.env`; value redacted and not included in this report.
`CLIENT_URL`	`http://localhost:5173` (allowed CORS origin)
`VITE_API_URL`	`http://localhost:5000/api`

Backend startup transcript

This is the verified output from the backend terminal session. It contains no environment values:

```
PS C:\Users\786\Downloads\LearnHub-MERN-Q2> Set-Location 'C:\Users\786\Downloads\LearnHub-MERN-
Q2\Authentication\backend'; npm run dev

> learnhub-backend@1.0.0 dev
> nodemon server.js

[nodemon] starting `node server.js`
MongoDB connected: 127.0.0.1
LearnHub API running on port 5000
```

Pretty-print

{"status":"ok","service":"LearnHub API"}

Figure 1. Live `GET /api/health` response captured from the running backend.

Application Screenshots

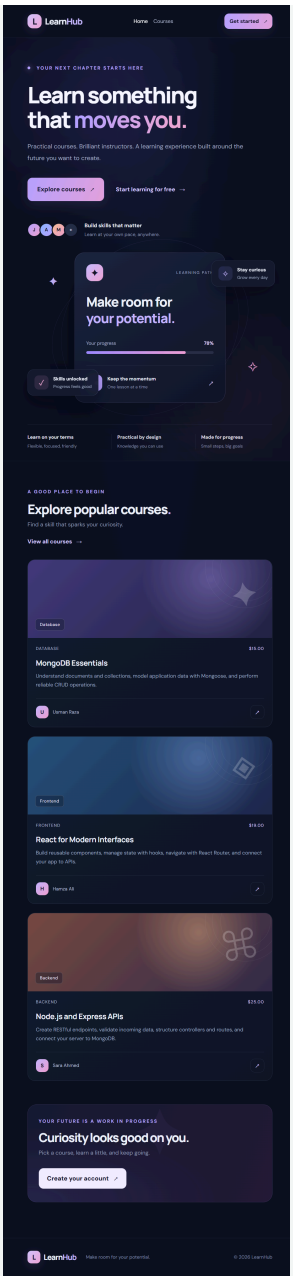


Figure 2. Public home page with the four seeded courses loaded from MongoDB.

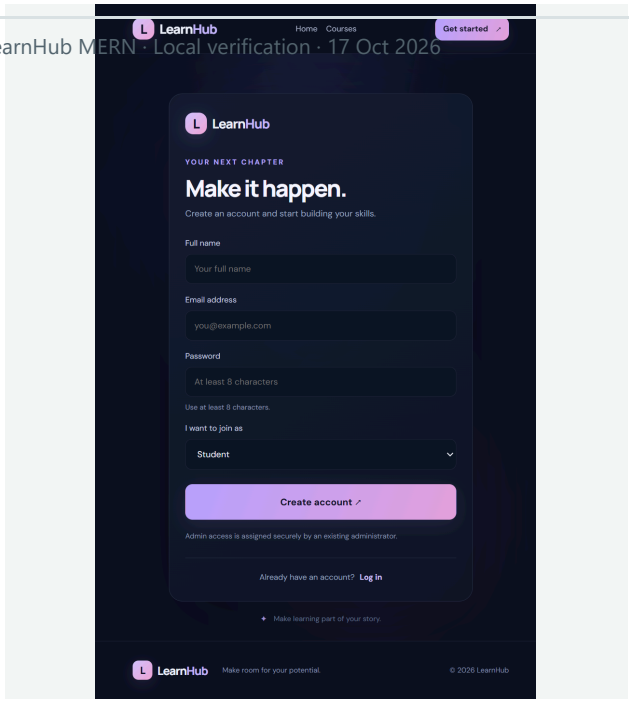


Figure 3. Registration form; public roles are Student and Instructor.

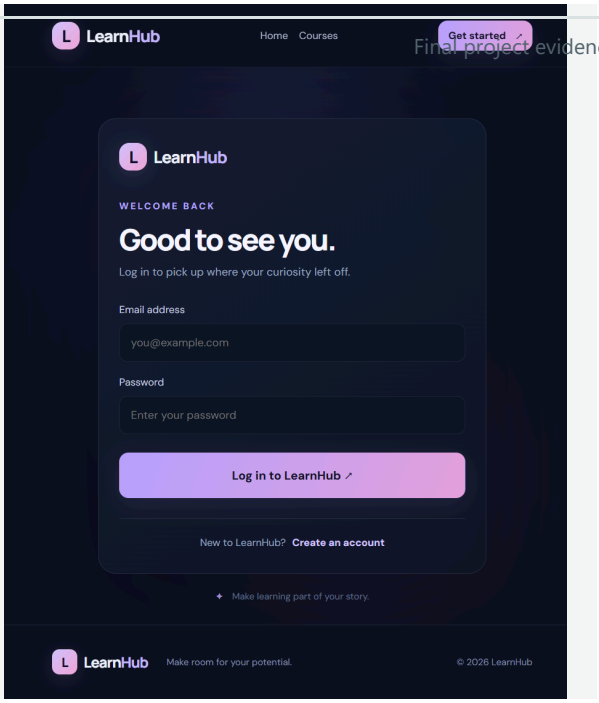


Figure 4. Login form with no credentials entered.

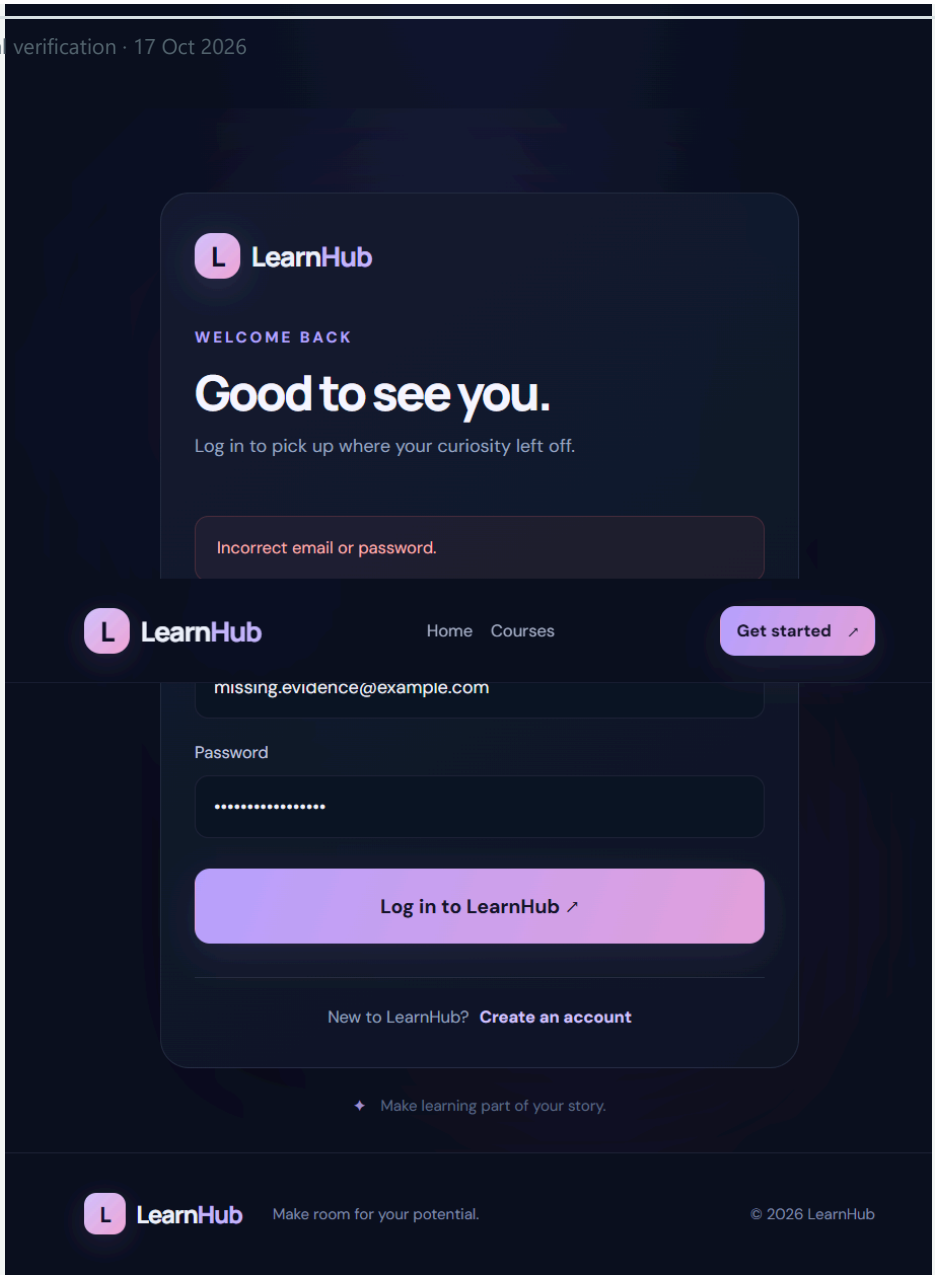


Figure 5. Real API 401 response rendered as invalid-credentials feedback. The entered password is masked and is not reproduced in this report.

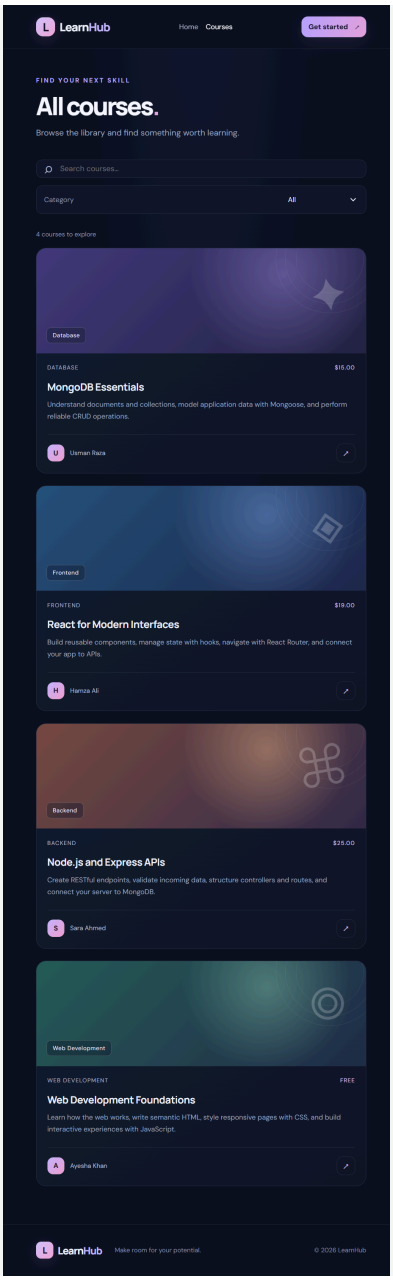


Figure 6. Course catalog after the instructor-name compatibility fallback; four database-backed courses are shown.

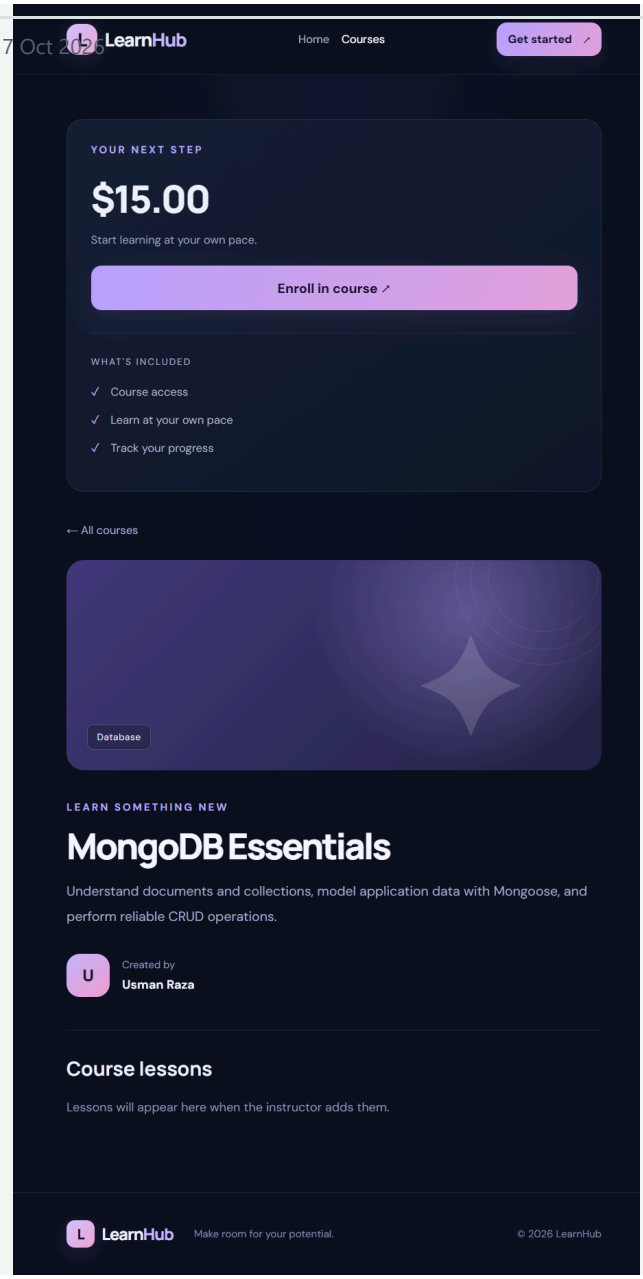


Figure 7. Course detail view showing the stored instructor name from the live API record.

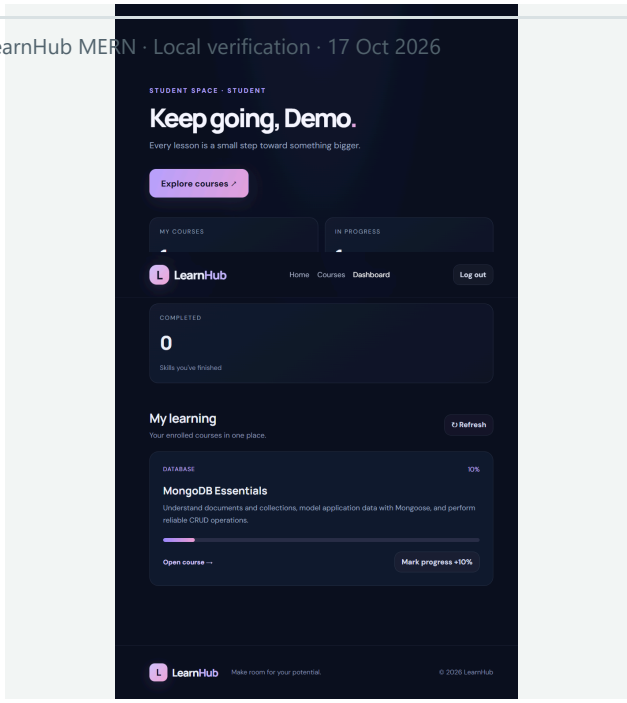


Figure 8. Student dashboard loaded from an authenticated session.

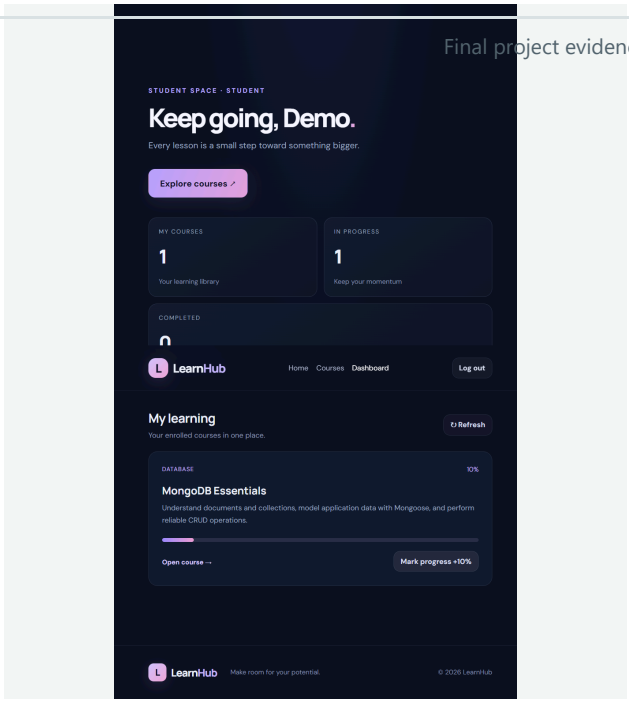


Figure 9. My Courses shows the enrolled MongoDB course and 10% progress. Captured before the temporary local test account was cleaned up.

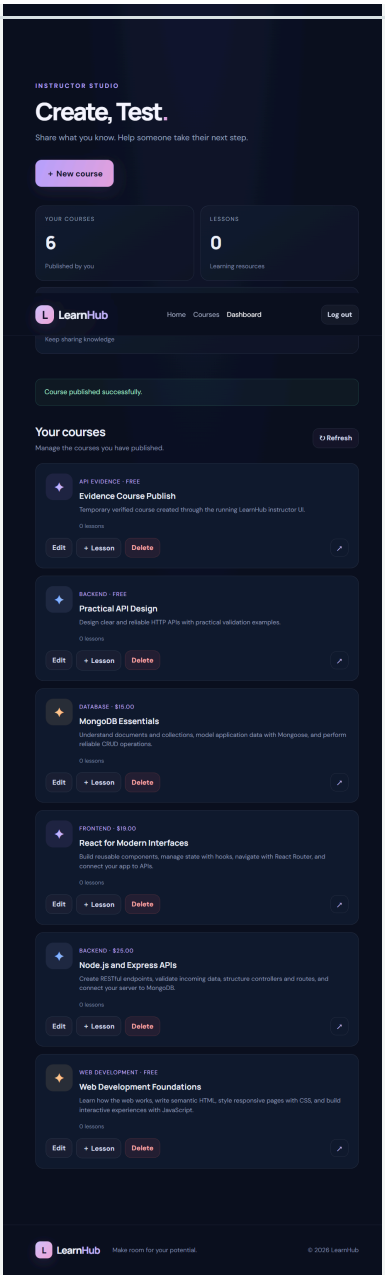


Figure 10. Instructor dashboard after a real course-create request succeeded; the temporary test course was removed after capture.

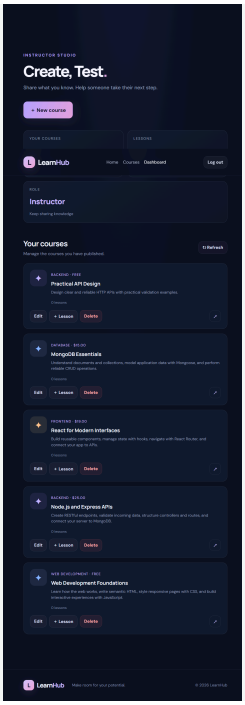


Figure 11. Instructor workspace with course-management actions.



Figure 12. Admin analytics, user management, and course library. Captured during authenticated admin verification.

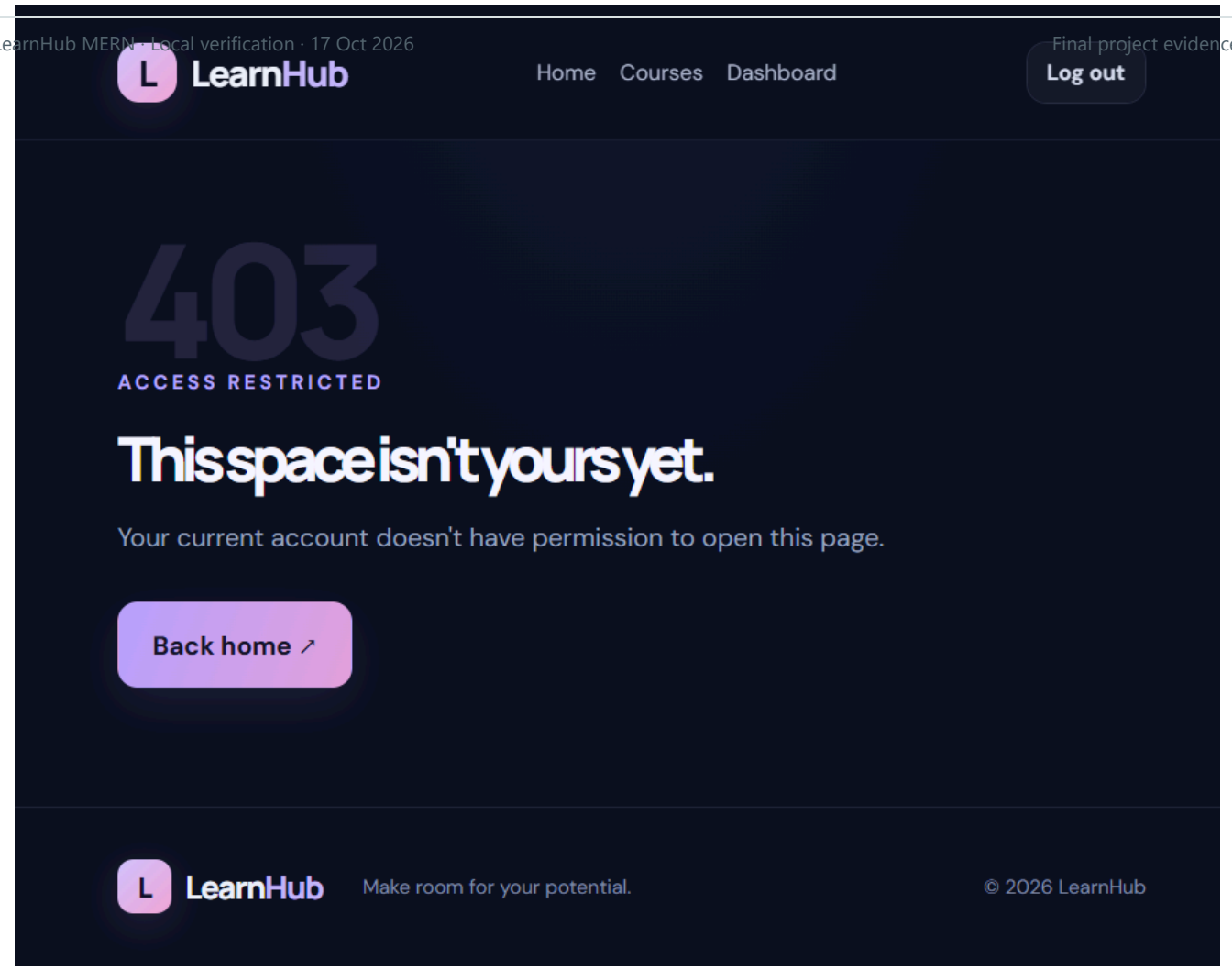


Figure 13. A logged-in Student attempting to open the Admin route is redirected to the actual 403 unauthorized page.

API Contract

LearnHub MERN · Local verification · 17 Oct 2026 Final project evidence

Base URL: `http://localhost:5000/api`. Protected calls require `Authorization: Bearer <token>`. The actual HTTP tests used PowerShell `Invoke-WebRequest/Invoke-RestMethod`; Postman and Newman were not installed. The request bodies below reflect the controllers and routes.

Method and endpoint	Access	Request body / notes
GET `/health`	Public	No body.
POST `/auth/register`	Public	`{ name, email, password: "<redacted>", role }`. Public registration permits student/instructor; an admin role request is downgraded to student.
POST `/auth/login`	Public	`{ email, password: "<redacted>" }`. Returns a JWT; token omitted here.
GET `/auth/me`	Bearer token	Current authenticated user.
GET `/courses`	Public	Optional `search` and `category` query parameters.
GET `/courses/:id`	Public	Single course by MongoDB ID.
POST `/courses`	Instructor/Admin	`{ title, description, category, price }`.
PUT `/courses/:id`	Owner/Admin	Any of `title, description, category, price`.
DELETE `/courses/:id`	Owner/Admin	No body; removes related enrollments and course.
POST `/courses/:id/lessons`	Owner/Admin	`{ title, url, description }`; URL must be HTTP(S).
POST `/enroll`	Student	`{ courseId }`.
GET `/enroll/my-courses`	Student	Returns the student's enrollments.
PATCH `/enroll/:id/progress`	Student/owner	`{ progress }`, numeric range 0–100.
GET `/admin/analytics`	Admin	No body.
GET `/admin/users`	Admin	No body; excludes password hashes.
PATCH `/admin/users/:id/role`	Admin	`{ role: "student" | "instructor" | "admin" }`.

HTTP Test Results

LearnHub MERN · Local verification · 17 Oct 2026

Final project evidence

All 24 checks below ran against the local API and passed their expected status-code assertions. The temporary course and users created for the API test were removed afterward; the database was checked to contain the original four courses.

Operation	Expected/actual HTTP	Result
GET `/health`	200	Pass
POST `/auth/register` as student	201	Pass
POST `/auth/register` as instructor	201	Pass
POST `/auth/login` as admin	200	Pass
GET `/auth/me` with bearer token	200	Pass
GET `/courses`	200	Pass
POST `/courses` as instructor	201	Pass
GET `/courses/:id`	200	Pass
PUT `/courses/:id` as owner	200	Pass
POST `/courses/:id/lessons`	201	Pass
POST `/enroll` as student	201	Pass
GET `/enroll/my-courses`	200	Pass
PATCH `/enroll/:id/progress`	200	Pass
Duplicate POST `/enroll`	409	Pass (expected conflict)
POST `/courses` as student	403	Pass (expected forbidden)
GET `/admin/analytics` as admin	200	Pass
GET `/admin/users` as admin	200	Pass
PATCH `/admin/users/:id/role` then restore role	200 / 200	Pass
GET `/admin/analytics` as student	403	Pass (expected forbidden)
GET `/admin/analytics` without token	401	Pass (expected unauthorized)
POST `/auth/login` with invalid credentials	401	Pass (expected unauthorized)
DELETE `/courses/:id` as owner	200	Pass
GET deleted `/courses/:id`	404	Pass (expected not found)

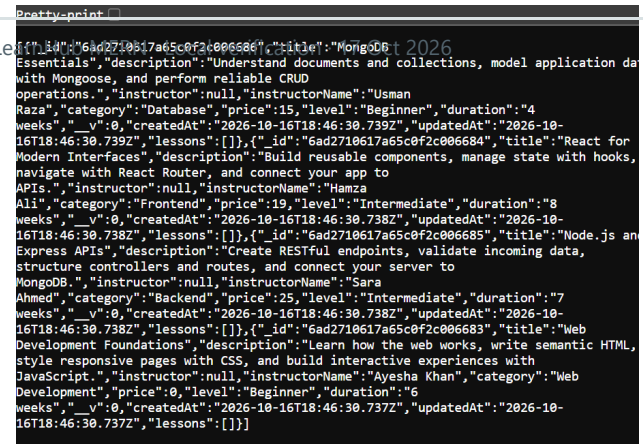


Figure 14. Actual JSON from `GET /api/courses` after temporary test data cleanup.

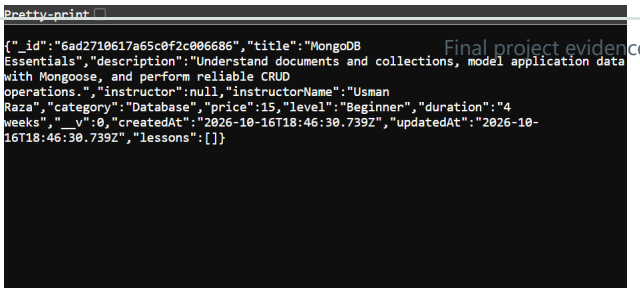


Figure 15. Actual JSON from `GET /api/courses/:id`; legacy record fields are visible.

Source Evidence

LearnHub MERN · Local verification · 17 Oct 2026

These images show the actual source files opened directly from the project. The backend contracts, token middleware, database configuration, frontend API client, and instructor fallback are visible. The local `.env` files were not opened or captured.

```
const router = require("express").Router();
const { getCourses, getCourse, createCourse, updateCourse, deleteCourse, addLesson }
= require("../controllers/courseController");
const { protect, allowRoles } = require("../middleware/auth");

router.get("/", getCourses);
router.get("/:id", getCourse);
router.post("/", protect, allowRoles("admin", "instructor"), createCourse);
router.put("/:id", protect, allowRoles("admin", "instructor"), updateCourse);
router.delete("/:id", protect, allowRoles("admin", "instructor"), deleteCourse);
router.post("/:id/lessons", protect, allowRoles("admin", "instructor"), addLesson);

module.exports = router;
```

Figure 16. Course routes and role restrictions.

```
const jwt = require("jsonwebtoken");
const User = require("../models/User");

async function protect(req, res, next) {
  try {
    const header = req.headers.authorization || "";
    const token = header.startsWith("Bearer ") ? header.slice(7) : null;
    if (!token) return res.status(401).json({ message: "Please log in to continue."
});
    if (!process.env.JWT_SECRET) return res.status(500).json({ message: "Server
authentication is not configured." });

    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(decoded.id).select("-password");
    if (!user) return res.status(401).json({ message: "Account not found. Please log
in again." });
    req.user = user;
    next();
  } catch (error) {
    return res.status(401).json({ message: "Your session is invalid or expired.
Please log in again." });
  }
}

function allowRoles(...roles) {
  return (req, res, next) => {
    if (!req.user || !roles.includes(req.user.role)) {
      return res.status(403).json({ message: "You do not have permission to perform
this action." });
    }
    next();
  };
}

module.exports = { protect, allowRoles };
```

Figure 17. JWT verification and role authorization middleware.

```
const asyncHandler = require("express-async-handler");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const User = require("../models/User");

function makeToken(userId) {
  if (!process.env.JWT_SECRET) throw Object.assign(new Error("JWT_SECRET is not
configured."), { statusCode: 500 });
  return jwt.sign({ id: userId }, process.env.JWT_SECRET, { expiresIn:
process.env.JWT_EXPIRES_IN || "7d" });
}

const register = asyncHandler(async (req, res) => {
  const { name, email, password, role } = req.body;
  if (!name || !email || !password) {
    res.status(400); throw new Error("Name, email, and password are required.");
  }
  if (String(name).trim().length < 2) {
    res.status(400); throw new Error("Name must contain at least 2 characters.");
  }
  const normalizedEmail = String(email).trim().toLowerCase();
  const validEmail = /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(normalizedEmail);
  if (!validEmail) { res.status(400); throw new Error("Enter a valid email
address."); }
  if (String(password).length < 8) {
    res.status(400); throw new Error("Password must be at least 8 characters long.")
  }
  if (await User.findOne({ email: normalizedEmail })) {
    res.status(409); throw new Error("An account with this email already exists.");
  }
  // Public registration never allows self-assigned admin access.
  const safeRole = role === "instructor" ? "instructor" : "student";
  const hashedPassword = await bcrypt.hash(password, 12);
  const user = await User.create({ name: String(name).trim(), email: normalizedEmail
password: hashedPassword, role: safeRole });
  res.status(201).json({
    token: makeToken(user._id),
    user: { id: user._id, name: user.name, email: user.email, role: user.role }
  });
});

const login = asyncHandler(async (req, res) => {
  const { email, password } = req.body;
  if (!email || !password) { res.status(400); throw new Error("Email and password ar
required."); }
  const user = await User.findOne({ email: String(email).trim().toLowerCase()
}).select("+password");
  if (!user || !(await bcrypt.compare(String(password), user.password))) {
    res.status(401); throw new Error("Incorrect email or password.");
  }
  res.json({
    token: makeToken(user._id),
    user: { id: user._id, name: user.name, email: user.email, role: user.role }
  });
});

const me = asyncHandler(async (req, res) => {
  res.json({ user: { id: req.user._id, name: req.user.name, email: req.user.email,
role: req.user.role } });
});

module.exports = { register, login, me };
```

Figure 18. Registration, password hashing, login, and token creation.

```
const mongoose = require("mongoose");

async function connectDB() {
  if (!process.env.MONGO_URI) {
    throw new Error("MONGO_URI is missing. Copy .env.example to .env and configure
it.");
  }
  const connection = await mongoose.connect(process.env.MONGO_URI);
  console.log(`MongoDB connected: ${connection.connection.host}`);
}

module.exports = connectDB;
```

Figure 19. Database connection reads the configured environment variable without exposing its value.

```
import axios from "axios";

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || "http://localhost:5000/api",
  headers: { "Content-Type": "application/json" },
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem("learnhub_token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

api.interceptors.response.use(
  response => response,
  error => {
    if (error.response?.status === 401 && localStorage.getItem("learnhub_token")) {
      // Keep the token so the user can read the error and choose to sign in again.
    }
    return Promise.reject(error);
  }
);

export const errorMessage = (error) =>
  error?.response?.data?.message || error?.message || "Request failed. Please try
again.";

export default api;
```

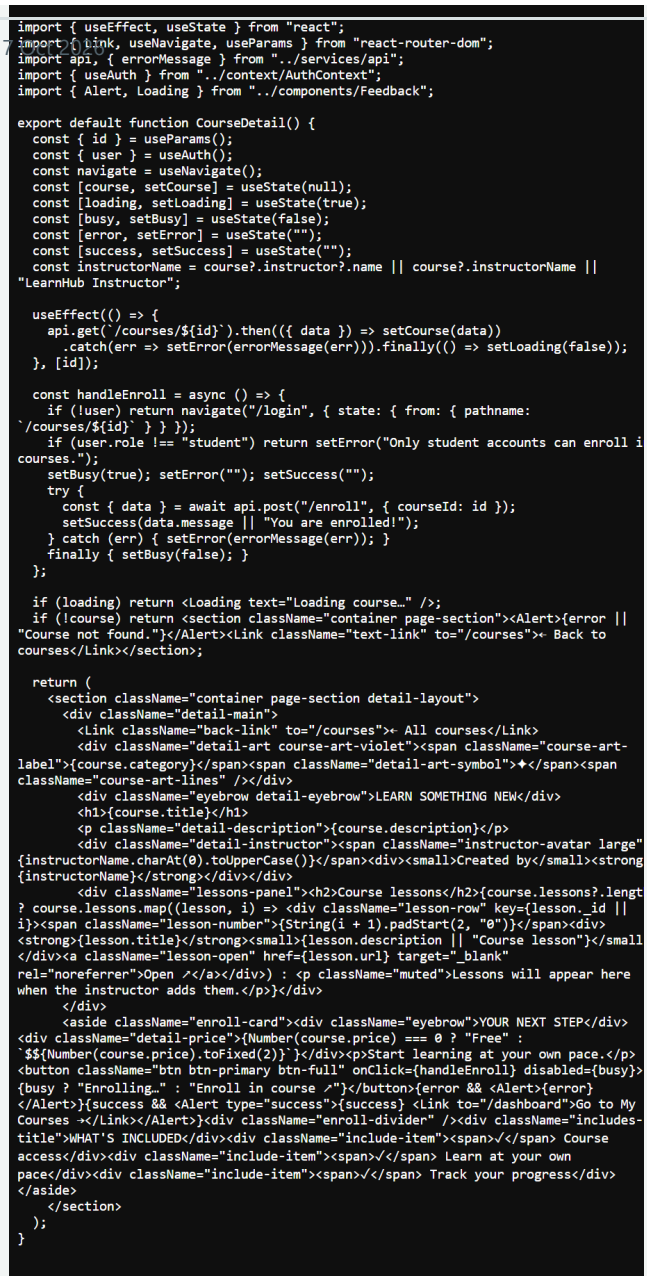
Figure 20. Axios base URL and bearer-token interceptor.

```
import { Link } from "react-router-dom";

const palettes = [
  "course-art-violet", "course-art-blue", "course-art-orange", "course-art-green"
];

export default function CourseCard({ course, index = 0 }) {
  const instructorName = course.instructor?.name || course.instructorName ||
"LearnHub Instructor";
  return (
    <article className="course-card">
      <div className={`course-art ${palettes[index % palettes.length]}`}>
        <span className="course-art-label">{course.category || "Learning"}</span>
        <span className="course-art-icon">{["✦", "◈", "⌘", "◎"][index % 4]}</span>
        <span className="course-art-lines" />
      </div>
      <div className="course-card-body">
        <div className="course-meta"><span>{course.category}</span><span>
{Number(course.price) === 0 ? "Free" : `$${Number(course.price).toFixed(2)}`}</span>
</div>
        <h3>{course.title}</h3>
        <p>{course.description}</p>
        <div className="course-card-footer">
          <span className="instructor-avatar">
{instructorName.charAt(0).toUpperCase()}</span>
          <span className="instructor-name">{instructorName}</span>
          <Link className="arrow-link" to={`/courses/${course._id}`} aria-label=
{`View ${course.title}`}>↗</Link>
        </div>
      </div>
    </article>
  );
}
```

Figure 21. Course card compatibility fallback for existing MongoDB records.



```
import { useEffect, useState } from "react";
import { Link, useNavigate, useParams } from "react-router-dom";
import api, { errorMessage } from "../services/api";
import { useAuth } from "../context/AuthContext";
import { Alert, Loading } from "../components/Feedback";

export default function CourseDetail() {
  const { id } = useParams();
  const { user } = useAuth();
  const navigate = useNavigate();
  const [course, setCourse] = useState(null);
  const [loading, setLoading] = useState(true);
  const [busy, setBusy] = useState(false);
  const [error, setError] = useState("");
  const [success, setSuccess] = useState("");
  const instructorName = course?.instructor?.name || course?.instructorName ||
"LearnHub Instructor";

  useEffect(() => {
    api.get(`/courses/${id}`).then(({ data }) => setCourse(data))
      .catch(err => setError(errorMessage(err))).finally(() => setLoading(false));
  }, [id]);

  const handleEnroll = async () => {
    if (!user) return navigate("/login", { state: { from: { pathname:
`/courses/${id}` } } });
    if (user.role !== "student") return setError("Only student accounts can enroll i
courses.");
    setBusy(true); setError(""); setSuccess("");
    try {
      const { data } = await api.post("/enroll", { courseId: id });
      setSuccess(data.message || "You are enrolled!");
    } catch (err) { setError(errorMessage(err)); }
    finally { setBusy(false); }
  };

  if (loading) return <Loading text="Loading course…" />;
  if (!course) return <section className="container page-section"><Alert>{error ||
"Course not found."}</Alert><Link className="text-link" to="/courses">← Back to
courses</Link></section>;

  return (
    <section className="container page-section detail-layout">
      <div className="detail-main">
        <Link className="back-link" to="/courses">← All courses</Link>
        <div className="detail-art course-art-violet"><span className="course-art-
label">{course.category}</span><span className="detail-art-symbol">✦</span><span
className="course-art-lines" /></div>
        <div className="eyebrow detail-eyebrow">LEARN SOMETHING NEW</div>
        <h1>{course.title}</h1>
        <p className="detail-description">{course.description}</p>
        <div className="detail-instructor"><span className="instructor-avatar large"
{instructorName.charAt(0).toUpperCase()}</span><div><small>Created by</small><strong
{instructorName}</strong></div></div>
        <div className="lessons-panel"><h2>Course lessons</h2>{course.lessons?.lengt
? course.lessons.map((lesson, i) => <div className="lesson-row" key={lesson._id ||
i}><span className="lesson-number">{String(i + 1).padStart(2, "0")}</span><div>
<strong>{lesson.title}</strong><small>{lesson.description || "Course lesson"}</small
</div><a className="lesson-open" href={lesson.url} target="_blank"
rel="noreferrer">Open ↗</a></div>) : <p className="muted">Lessons will appear here
when the instructor adds them.</p>}</div>
      </div>
      <aside className="enroll-card"><div className="eyebrow">YOUR NEXT STEP</div>
<div className="detail-price">{Number(course.price) === 0 ? "Free" :
`$${Number(course.price).toFixed(2)}`}</div><p>Start learning at your own pace.</p>
<button className="btn btn-primary btn-full" onClick={handleEnroll} disabled={busy}>
{busy ? "Enrolling…" : "Enroll in course ↗"}</button>{error && <Alert>{error}
</Alert>}{success && <Alert type="success">{success} <Link to="/dashboard">Go to My
Courses →</Link></Alert>}<div className="enroll-divider" /><div className="includes-
title">WHAT'S INCLUDED</div><div className="include-item"><span>✓</span> Course
access</div><div className="include-item"><span>✓</span> Learn at your own
pace</div><div className="include-item"><span>✓</span> Track your progress</div>
</aside>
    </section>
  );
}
```

Figure 22. Course detail instructor name uses the populated relation first, then the legacy stored display field.

Conclusion and Submission Notes

The local application is running and the tested API/UI flows passed. The core paths verified include registration/login, JWT checks, role-specific dashboards, protected routes, course CRUD, lessons, student enrollment, progress, admin role changes, and expected error responses.

Evidence item	Outcome
Local frontend/backend/database	Verified live at `localhost:5173`, `localhost:5000`, and local MongoDB.
Frontend production build	Pass; Vite emitted a non-fatal chunk-size warning.
Postman	Not installed. API testing used PowerShell HTTP requests against the real API, not a simulated collection run.
Backend terminal image	Not included; the active backend log was verified but the desktop capture remained on a different VS Code terminal. A genuine API health screenshot and safe startup transcript are included instead.
Deployment	Not performed; no hosted URL is claimed.
Credentials/secrets	No passwords, JWTs, or environment-variable values are included.

The bundled Postman collection contains only a subset of implemented endpoints; the live test matrix above also verifies lesson creation, progress updates, role changes, and authorization failures. Before deployment, review dependency audit findings and validate the ownership references on legacy seed records if instructors need to edit those specific courses.

Run locally: follow `README.md`. Admin access must be assigned to a trusted account in MongoDB; public registration intentionally cannot grant admin privileges.